

0.30. Un pas sans attendre : un_pas

un_pas fait UN pas, tout de suite, sans bloquer. C'est la boucle qui donne le rythme.

1. Ce qu'il faut comprendre

Le défaut des fonctions d'avant : `deplace_x`, `deplace_y`, `deplace` et `va_a` **bloquent**. Pendant le voyage, le programme attend : rien d'autre ne peut bouger.

Ce qui est nouveau ici : `un_pas(colonne, ligne, tuile, sensX, sensY)`. Elle fait **un seul pas**, tout de suite, et rend la main aussitôt : elle efface la lettre et la pose une case plus loin. Elle **n'attend pas**.

Seul le signe compte : `1` vers la droite (X) ou le bas (Y) ; `-1` vers la gauche ou le haut ; `0` ne bouge pas sur cet axe.

C'est la boucle qui donne le rythme, avec le chronomètre du 0.17 : `images` compte les images, et tous les 15 (quatre fois par seconde), on fait un pas.

Au bord de l'écran (colonne 19), un pas vers l'extérieur ne fait plus bouger la lettre.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1616 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;          // la colonne de la lettre
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;          // sa ligne
```

3. UNE VARIABLE — IMAGES

```
uint8_t images = 0;     // le chronomètre, en images (comme au 0.17)
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(x, y, ALPHABET[0]);    // la lettre à sa place de départ

    while (true) {
        image();                 // attend l'image suivante (60 par seconde)
        images++;               // le chronomètre avance d'une image
        if (images == 15) {     // 15 images : un quart de seconde
            images = 0;         // le chronomètre repart de zéro

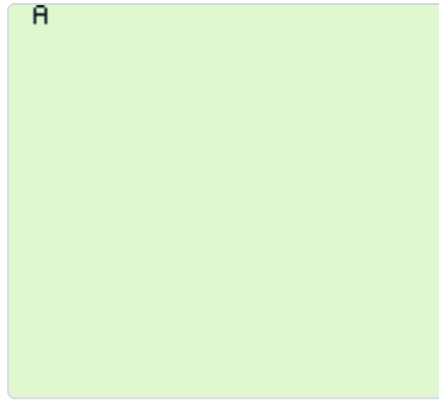
            // La ligne, morceau par morceau :
            //
            // un_pas(x, y, ALPHABET[0], 1, 0);
            //      | | |           | |
            //      | | |           | +-- sensY : 0, ne bouge pas sur Y
            //      | | |           +----- sensX : 1, un pas vers la droite
            //      | | +----- tuile : la lettre A
            //      +----- où elle est ; la console y range
            //                      sa nouvelle place
            //
            // un_pas N'ATTEND PAS : elle fait le pas et rend la main tout de suite.
            un_pas(x, y, ALPHABET[0], 1, 0);
        }
    }
}
```

3. Ce qui se passe, étape par étape

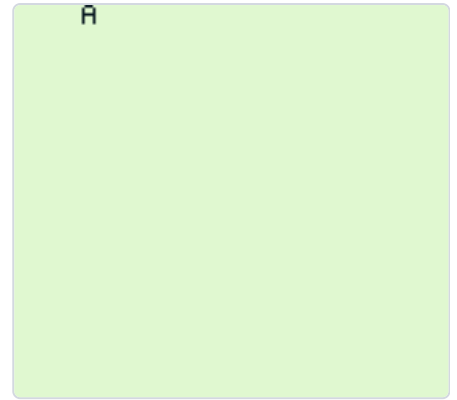
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



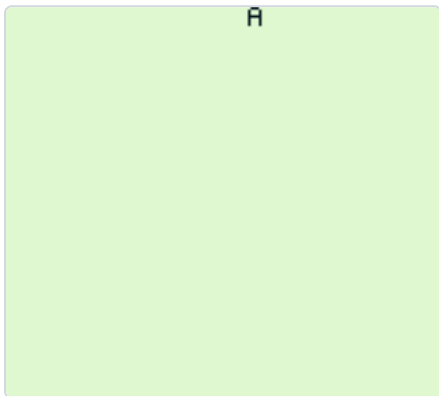
la toute première image — la console vient de démarrer — image 1



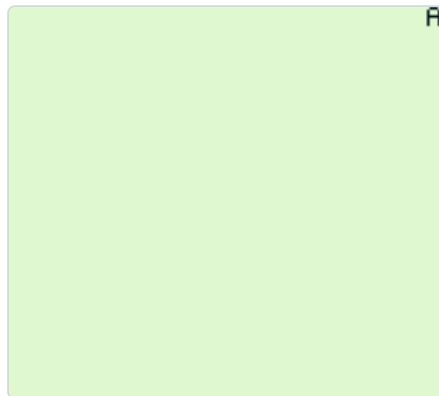
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui avance d'une case tous les quarts de seconde vers la droite, et s'arrête au bord.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ un pas tous les quarts de seconde : 3 ou 4 en une seconde (4)
- ✓ le A est à sa place à l'écran
- ✓ il s'arrête au bord : colonne 19 ($x = 19$)

Ensuite : **0.30.1. Un pas sans attendre : un_pas — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++